

Projeto e Análise de Algoritmos

Classes de Complexidade

Thiago Pinheiro de Araújo

EMAp/FGV

2025.2

Apresentar a classificação de problemas com base na sua complexidade e enunciar a questão $P \neq NP$.

Motivação

- A **complexidade computacional** de um problema é definida pela complexidade do algoritmo com maior eficiência capaz de resolvê-lo.
- **Importante:** o melhor algoritmo para resolver um determinado problema é o melhor algoritmo conhecido para ele
 - O melhor algoritmo possível pode ainda não ter sido descoberto!
- Dado um problema, buscamos criar para ele algoritmos cada vez mais rápidos
 - No entanto, existem problemas cuja complexidade ainda é considerada alta.
 - Após anos de tentativas frustradas, é natural levantar a suspeita que tal algoritmo mais eficiente não exista.

Motivação

- Essa situação ocorre mesmo para problemas que possuem um problema semelhante com algoritmo polinomial conhecido.
- Exemplos:
 - Caminho simples e mais curtos e mais longos.
 - Viagem de Euler e Ciclo Hamiltoniano.
 - 2-SAT e 3-SAT.
- Vamos buscar responder as perguntas:
 - O que define um algoritmo como rápido?
 - Quando é razoável esperar um algoritmo rápido de um problema?

Motivação

- Um problema é **tratável** caso possua um algoritmo capaz de ser executado em tempo polinomial
 - Conseguem tratar entradas grandes com recursos computacionais razoáveis.
- Um problema é **intratável** caso o algoritmo mais eficiente exija tempo superpolinomial (exponencial, fatorial, etc)
 - Exigem uma quantidade de tempo ou recursos impraticável.
 - Podem ter algoritmos baseados em aproximações ou heurísticas.
- Um problema é **indecidível** caso não exista um algoritmo capaz de fornecer uma resposta correta para todas as possíveis entradas.

Tipos de problemas

- Consideramos um **problema de otimização** quando sua solução possui um valor associado, e buscamos encontrar a solução com melhor valor
 - Ex: encontrar um ciclo simples de comprimento máximo em um grafo.
- Consideramos um **problema de busca** quando a solução possui um valor com um conjunto de propriedades
 - Ex: encontrar um ciclo simples de comprimento maior ou igual à k em um grafo.
- Consideramos um **problema de decisão** quando a solução consiste em um sim/não para uma pergunta
 - Ex: decidir se o grafo possui um ciclo simples de comprimento maior ou igual à k .

Tipos de problemas

- Observe que um problema de busca não é mais difícil que um problema de otimização.
- Observe também que um problema de decisão não é mais difícil que um problema de busca.
- Podemos inclusive construir um algoritmo de decisão utilizando o algoritmo de otimização do problema correspondente.
- Portanto, se conseguirmos demonstrar que um algoritmo de decisão não possui um algoritmo tratável, demonstramos que o correspondente algoritmo de otimização também não tem.

- Um **problema abstrato** é definido por um conjunto de instâncias $i \in I$ e uma função $f(i)$ que mapeia cada instância i em uma solução $s \in S$.
- Para um computador resolver um problema abstrato as instâncias devem ser submetidas à uma **codificação** utilizando um alfabeto Σ
 - Considere o alfabeto $\Sigma = \{0,1\}$.
- Um algoritmo resolve um **problema concreto** em $O(T(n))$ se ao fornecer uma instância i com tamanho $n = |i|$ o algoritmo produzir a solução em no máximo $O(T(n))$
 - Atingimos isso considerando um custo de operação polinomialmente equivalente à máquina de Turing.

A classe P

- Um algoritmo resolve um problema em **tempo polinomial** se o seu consumo de tempo no pior caso for limitado por uma função polinomial com base nos tamanhos das instâncias.
 - Ou seja, se existe uma constante k tal que o consumo de tempo seja $O(n^k)$.
- Embora não pareça ser razoável definir como tratável um problema que exija um tempo $\Theta(n^{100})$ na prática existem poucos algoritmos com um grau tão alto
 - Historicamente, quando um algoritmo é descoberto para um problema, frequentemente algoritmos mais eficientes são descobertos em seguida.
- A **classe P** é definida pelos problemas de decisão tratáveis
 - Ou seja, os que possuem um algoritmo com tempo de execução polinomial.

- Um problema é razoável se conseguirmos reconhecer a solução de uma instância de forma eficiente.
- Um **verificador** para um problema de decisão A consiste em um algoritmo que recebe uma instância i e um certificado C , e responde SIM sempre que C for uma solução de A para i .
- Exemplo: dado um grafo $G = (V, E)$ verifique se existe um ciclo hamiltoniano
 - Para esse problema o certificado seria a lista de vértices que representa um ciclo contendo todos os vértices.

A classe NP

- A **classe NP** (*nondeterministic polynomial*) é definida pelos problemas de decisão que possuem um verificador cujo algoritmo seja capaz de executar em tempo polinomial para toda resposta SIM
 - Atenção: isso não implica que o problema possa ser resolvido em tempo polinomial, apenas que a sua solução possa ser verificada de forma eficiente!
- Dado um problema α observe que se $\alpha \in P$, então $\alpha \in NP$
 - Se existe um algoritmo A com tempo polinomial para decidir α , é possível criar um algoritmo de verificação A_v que faça uso de A para realizar a verificação em tempo polinomial.
- Portanto, temos que $P \subseteq NP$.

A questão $P \neq NP$

- Não se sabe se $P = NP$ ou $P \neq NP$
 - *Todos os problemas cujas soluções podem ser verificadas em tempo polinomial poderiam também ser resolvidos em tempo polinomial?*
- Trata-se de um dos maiores problemas de pesquisa em aberto na área da ciência da computação
 - O problema foi proposto em 1971!
- Acredita-se que $P \neq NP$, no entanto ninguém conseguiu provar ainda
 - Uma linha de argumentação: é mais difícil resolver um problema do zero à verificar se uma solução está correta.

Reduções de problemas

- Podemos utilizar o conceito de **redução polinomial** para mostrar que um problema é pelo menos tão difícil quanto outro.
- Seja α um problema de decisão que desejamos resolver em tempo polinomial.
- Seja β um problema de decisão que conhecemos um algoritmo capaz de resolver qualquer instância em tempo polinomial.
- Se conseguirmos criar uma transformação f que utilize o problema β para resolver o problema α , e f seja executada em tempo polinomial, teremos criado um algoritmo para resolver α em tempo polinomial.

Reduções de problemas

- Portanto, dizemos que um problema de decisão α é polinomialmente redutível a um problema de decisão β ($\alpha \leq \beta$) se
 - Existe um algoritmo polinomial T que transforma qualquer instância i_α de α em uma instância $i_\beta = T(i_\alpha)$ de β ; e
 - i_β tem solução SIM se e somente se i_α tem solução SIM.
- **Exemplo:** dada uma lista A de tamanho n , o i -ésimo menor elemento de A é x ?
 - Como resolver esse problema sem utilizar um algoritmo de seleção diretamente?
 - Solução:
 - Criar um algoritmo que ordena a lista e retorna o resultado de $A[i] == x$.

Classe NP-Completo

- Não sabemos se existe um algoritmo polinomial para muitos problemas computacionais de interesse prático
 - Muitos possuem um algoritmo de verificação polinomial.
 - Buscamos então entender a dificuldade desses problemas.
- Um problema é **NP-Difícil** se todos os problemas da classe NP forem polinomialmente redutíveis à ele
 - Ou seja, é tão difícil quanto qualquer problema em NP.
- Um problema é **NP-Completo** se for NP-Difícil e estiver em NP.

Classe NP-Completo

- O primeiro problema NP-Completo foi descoberto em 1971 por Cook e Levin.
- Utilizando o conceito de redução polinomial podemos demonstrar que um determinado problema α é NP-Completo:
 - Se β é NP-Completo e $\beta \leq \alpha$, temos que α também é NP-Completo.
- Observe que se algum problema NP-Completo puder ser resolvido em tempo polinomial, teremos $P = NP$.

Perguntas?

`thiago.p.araujo@fgv.br`